

Project A1

Freight Cost Intelligence

Health Commodity Freight Cost Prediction Enriched with UNCTAD Bilateral Rate
Benchmarks

USAID Transaction Data + UNCTAD-World Bank Global Transport Costs Dataset + Unified
RAG + What-If Chat

Sameer | Habib University, Karachi

Portfolio Block 1 — April to July 2026

Part of a 4-project arc targeting supply chain data science roles in Pakistan
and later RWTH Aachen MSc Data Science application (Winter 2028/29)

v4 — Pivoted from two-dataset multimodal to USAID + UNCTAD enrichment after Brunel EDA gate
failure (single-date snapshot, 99.5% AIR imbalance, 24% unratable orders)

Contents

1	Why This Project Exists	2
2	Project Specification	3
2.1	Problem Statement	3
2.2	App Features (Final State)	3
2.3	Datasets	4
2.4	Stack	5
3	Repository Structure	5
3.1	Copy-Paste Files	6
3.1.1	.gitignore	6
3.1.2	requirements.txt	6
3.1.3	.env.example	7
4	13-Week Roadmap	7
4.1	Phase 1 — UNCTAD EDA + USAID Feature Engineering + Baseline (Weeks 2–4)	7
4.2	Phase 2 — UNCTAD Feature Analysis + Buffer (Weeks 5–6)	8
4.3	Phase 3 — RAG Layer (Weeks 7–9)	9
4.4	Phase 4 — What-If Chat and App (Weeks 10–12)	10
4.5	Phase 5 — Ship (Week 13)	11
5	Alternatives Table	12
6	Risk Flags	14
7	What NOT To Do	14
8	Success Criteria	14

Why This Project Exists

One-line goal: Build and deploy a health commodity freight cost prediction model on USAID shipment data, enriched with UNCTAD bilateral freight rate benchmarks as external features, with generative SHAP-grounded explanation, cited trade-doc context, and a what-if chat — proving end-to-end SC-DS + LLM integration capability for Pakistan junior analyst roles and RWTH/Werkstudent applications.

This is not a Kaggle competition. The goal is a shipped, deployed artifact that a recruiter can click, a hiring manager can interrogate, and an RWTH admissions committee reads as a concrete portfolio piece.

Why supply chain? Supply chain was chosen over generic DS and fintech as the primary career vertical. It pairs domain specificity with genuine demand in Pakistan's logistics, e-commerce, and FMCG sectors. Junior analyst roles at companies like Daraz, Nestlé, and 3PLs explicitly look for cost modelling and demand forecasting skills.

Why generative explanation + chat? A plain prediction number has zero interview story. A prediction with a cited plain-English explanation — grounded in the model's SHAP values and retrieved trade documents — is memorable, differentiated, and demonstrates LLM integration, which is increasingly expected in SC-DS roles. The what-if chat upgrades the tool from demo to decision-support, which is the actual job.

Why USAID + UNCTAD enrichment, not two separate models? The v3 plan included Brunel Supply Chain Logistics as a second dataset. Brunel EDA (Week 2) revealed fatal structural problems: all 9,215 orders share a single date (no time-based split possible), 99.5% of ratable orders are AIR (GROUND sub-model has 34 rows), and 24% of orders are unratable due to a discontinued carrier. Brunel is a linear-programming benchmark snapshot, not a freight cost time series. It was dropped at the Week 2 decision gate.

The replacement enrichment strategy uses the **UNCTAD-World Bank Global Transport Costs Dataset for International Trade (GTCBIT)**, a free, authoritative dataset covering bilateral freight rates (air, sea, rail, road) by country pair and commodity group, 2016–2021. This is used not as a second training dataset but as a **lookup feature table**: for each USAID shipment, we join the UNCTAD ad-valorem freight rate benchmark for the relevant origin country → destination country → mode combination. This gives the model an external structural cost signal — the kind of benchmark a real freight analyst would consult. The framing in the README is accurate and defensible: “We use USAID transaction data enriched with UNCTAD bilateral freight rate benchmarks as a lane-level cost signal.”

Temporal mismatch — know this for interviews. USAID spans approximately 2006–2015; UNCTAD covers 2016–2021. There is no year overlap. UNCTAD is therefore used as a **structural benchmark**, not a time-matched join. The assumption: bilateral freight rate intensities for a given lane and mode are sufficiently stable structural signals even when drawn from a different period. This is standard practice in gravity-model freight research. In the README and in interviews, state this explicitly. Do not pretend the join is temporal.

How this fits the larger plan:

- Phase 1 (now): Build portfolio, GitHub, LinkedIn presence.
- Phase 2 (4th year + 1yr post-grad): Land SC data analyst/DS role in Pakistan (80–150k PKR).
- Phase 3 (mid-2027): GRE, IELTS, APS certificate, blocked account (EUR 11,208).
- Phase 4: RWTH Aachen MSc Data Science (Winter 2028/29) → Werkstudent → found company.

Seeds the final-year project (SC data + RAG/ML + deployed tool = triple asset: portfolio, thesis demo, interview piece).

Project Specification

Problem Statement

Build a freight cost prediction model on USAID health commodity shipment data (Air/Truck/Ocean, 10k+ records), enriched with UNCTAD bilateral freight rate benchmarks as engineered features. The model returns a predicted cost plus uncertainty range. A generative explanation layer reads SHAP values and retrieves relevant trade-document chunks to produce a plain-English summary with feature breakdown and cited context. A what-if chat lets the user ask follow-ups including scenario modifications (“what if I switch mode?”, “what if destination is Nigeria instead?”) that rerun the model and return a new prediction with explanation.

App Features (Final State)

Tab 1 — Predict and Explain

1. User fills shipment form: origin hub, destination country, mode (Air/Truck/Ocean), weight, commodity, INCO term.
2. Model returns predicted freight cost + uncertainty range.
3. UNCTAD benchmark for the lane is shown alongside: “UNCTAD ad-valorem rate for [origin] → [destination] via [mode]: X%”.
4. LLM generates plain-English summary of why this cost was predicted.
5. Feature importance breakdown table shown (top 5–7 SHAP drivers), including UNCTAD feature if it appears.
6. Cited trade-doc context shown (e.g. relevant Incoterms clause, HS code note).

Tab 2 — What-If Chat

1. Chat pre-seeded with current shipment context from Tab 1.
2. User asks alternatives: “What if I use sea instead of air?” “What if I ship to Uganda instead of Kenya?”
3. System parses intent → modifies input dict (including re-looking up UNCTAD benchmark for new lane) → reruns model → returns new prediction.
4. Response includes: new cost, delta vs original, plain-English explanation, cited trade docs.
5. General trade questions (Incoterms, HS codes, glossary) answered via unified RAG with citations.

Datasets

Dataset	Description
USAID Supply Chain Shipment Pricing	10k-Kaggle (apoorvwatsky / divyeshardeshana) real ship- ment records: ori- gin (hub), des- ti- na- tion coun- try, mode (Air dom- i- nant, Truck, some Ocean, N/A), weight, freight cost, INCO term, com- mod- ity. Do- main: HIV/ARV health com- modi- ties to PEP- FAR coun- tries. Time range: ap- prox. 2006- 2015. Used as pri- mary train

Do not commit raw data to git. Add `data/raw/` and `data/processed/` to `.gitignore`. Corpus PDFs in `data/corpus/` can be committed if under 10MB total.

On the UNCTAD enrichment design: UNCTAD GTCDIT is a lookup table, not a training dataset. For each USAID shipment row, we look up the UNCTAD ad-valorem freight rate for (origin_country, destination_country, mode) and add it as a feature column (`unctad_adval_rate`). If no match exists for a specific HS code, aggregate to commodity group or fall back to mode-level average for that country pair. Missing coverage → NaN → LightGBM handles natively. This produces one model, one training dataset, one deployment — significantly simpler architecture than the two-model plan.

Stack

Layer	Tools
Data wrangling	<code>pandas</code> , <code>numpy</code>
Modelling	<code>scikit-learn</code> (pipelines, baseline), <code>xgboost</code> , <code>lightgbm</code>
Explainability	<code>shap</code> — SHAP values for feature importance, feeds LLM explanation prompt
Visualization	<code>matplotlib</code> , <code>seaborn</code>
Embeddings	<code>sentence-transformers</code> (<code>all-MiniLM-L6-v2</code>) — CPU-safe
Vector store	<code>faiss-cpu</code> (primary) <code>chromadb</code> (alt)
PDF parsing	<code>pypdf</code> (primary) <code>pdfplumber</code> (alt)
LLM client	<code>groq</code> (free, primary) <code>openai</code> (fallback via env var)
App	<code>streamlit</code>
Deployment	Hugging Face Spaces (free)
Secrets	<code>python-dotenv</code> local, HF Secrets in prod
Tests	<code>pytest</code>
Linting	<code>ruff</code>

Key addition: `shap`. SHAP values are the bridge between the ML model and the LLM explanation. The top SHAP contributors for a given prediction get injected into the LLM prompt, grounding the explanation in actual model reasoning rather than hallucination. If `unctad_adval_rate` has high SHAP importance, the explanation can cite it as “the UNCTAD benchmark rate for this lane was X%, signalling a high-cost corridor.”

Hardware: 16GB RAM, Intel HD only. All choices are CPU-safe. Do not use CUDA-dependent models.

Repository Structure

```
freight-cost-rag/
|-- README.md
|-- requirements.txt
|-- .gitignore
|-- .env.example
|-- data/
|   |-- raw/
|   |   |-- usaid/          # SCMS_Delivery_History CSV
|   |   |-- unctad/        # GTCDIT bulk download CSV(s)
|   |-- processed/        # gitignored
|   |-- corpus/           # RAG PDFs -- commit if small
|-- notebooks/
|   |-- 01_eda_usaid.ipynb
|   |-- 02_eda_unctad.ipynb # schema tour, coverage audit, join plan
|   |-- 03_feature_engineering.ipynb
|   |-- 04_baseline.ipynb
|   |-- 05_tree_models.ipynb
```

```
| |-- 06_unctad_feature_analysis.ipynb # SHAP analysis of UNCTAD feature
| |-- 07_rag_prototype.ipynb
|-- src/
| |-- __init__.py
| |-- config.py
| |-- data/
| | |-- __init__.py
| | |-- load_usaid.py
| | |-- load_unctad.py # loads GTCDIT, builds lookup dict
| |-- features/
| | |-- __init__.py
| | |-- features.py # USAID features + UNCTAD enrichment join
| |-- models/
| | |-- __init__.py
| | |-- train.py
| | |-- predict.py
| |-- explainer.py # SHAP + LLM explanation
| |-- rag/
| | |-- __init__.py
| | |-- ingest.py
| | |-- retriever.py
| | |-- generator.py
| | |-- whatif.py
|-- tests/
| |-- test_features.py
| |-- test_unctad_join.py # verify coverage, fallback logic
| |-- test_explainer.py
| |-- test_rag.py
|-- app/
| |-- streamlit_app.py
|-- models/ # gitignored
| |-- usaid_model.pkl
|-- scripts/
| |-- train.py
```

Copy-Paste Files

.gitignore

```
__pycache__/
*.pyc
.env
.venv/
venv/
data/raw/
data/processed/
models/
*.ipynb_checkpoints/
.DS_Store
.vscode/
.idea/
*.log
.streamlit/secrets.toml
```

requirements.txt

```
pandas>=2.0
numpy
scikit-learn>=1.3
xgboost
lightgbm
shap
matplotlib
seaborn
jupyter
sentence-transformers
faiss-cpu
pypdf
groq
```

```
streamlit
python-dotenv
pytest
ruff
```

.env.example

```
GROQ_API_KEY=your_key_here
OPENAI_API_KEY=optional_fallback
LLM_PROVIDER=groq
```

13-Week Roadmap

Schedule: Weekend-heavy. Saturday/Sunday = 4–5hr deep work. Weekdays = 30–45min touches. Do not fight this rhythm.

Status at time of writing (v4): Week 1 USAID EDA complete. Brunel dropped at Week 2 decision gate. Plan restarts at Week 2 with UNCTAD EDA. USAID findings already documented: time-based 80/20 split at Dec 2013; weight = commodity weight (unit × qty); 5 outliers need 99th-pct capping; freight costs doubled over dataset period; log-log weight vs cost shows power-law relationship; CIF/DAP/DDU grouped as “Other”.

Phase 1 — UNCTAD EDA + USAID Feature Engineering + Baseline (Weeks 2–4)

Week 2 — UNCTAD Pull and EDA (~7 hrs)

Download:

- Go to <https://unctadstat.unctad.org/datacentre/dataviewer/US.TransportCosts>
- Select: all origins, all destinations, mode = air + sea (the two modes relevant to USAID), years = all available (2016–2021), indicator = ad-valorem transport costs (%) and per-unit transport costs (\$/kg).
- Download bulk CSV → `data/raw/unctad/`.

Weekday touches — `src/data/load_unctad.py`:

- Load GTCDIT CSV. Understand columns: origin (M49 code), destination (M49 code), commodity (HS 2017 4-digit), mode, year, ad-valorem rate, \$/kg rate.
- Map M49 country codes to ISO-3 or country names to match USAID destination country column.
- Build a lookup dict: `(origin_country, dest_country, mode) → median ad-valorem rate across years`. Median across 2016–2021 gives a stable structural benchmark.
- Check: what share of USAID origin/destination/mode combinations have UNCTAD coverage?

Weekend block — `02_eda_unctad.ipynb`:

- Coverage audit: for USAID rows, what % get a direct UNCTAD match? What % require country-level aggregation fallback? What % remain NaN?
- Distribution of UNCTAD ad-valorem rates for USAID lanes. Are the rates plausible (air should be higher than sea)?
- Correlation: does UNCTAD benchmark correlate with USAID actual freight cost at the lane level?
- “Findings” markdown cell: honest read on UNCTAD coverage and signal quality.

Week 2 decision gate for UNCTAD: If coverage is below 30% of USAID rows even after fallback, or if UNCTAD ad-valorem rates show zero correlation with USAID freight costs, drop the UNCTAD feature. Reframe to USAID-only. Do not sunk-cost a weak external feature. Post findings before proceeding.

Week 3 — Feature Engineering (~7 hrs)

Weekday touches:

- `src/data/load_usaid.py` — clean loader (outlier capping, mode grouping, log target).
- `src/features/features.py` — route encoding, weight bucketing, mode handling, INCO term encoding, year feature, UNCTAD join logic with fallback hierarchy.
- `tests/test_features.py` — sanity checks.
- `tests/test_unctad_join.py` — verify fallback logic: HS4 → HS2 → mode-level average → NaN.

Weekend block — 03_feature_engineering.ipynb:

- Final feature set with UNCTAD column included.
- Missing value audit post-join. How many NaNs in `unctad_adval_rate`?
- Time-based split: 80% train (pre-Dec 2013) / 20% test (post-Dec 2013).
- README update: problem, data, enrichment approach.

Week 4 — Baseline + Tree Models (~7 hrs)

04_baseline.ipynb:

- Log-transform target. LinearRegression baseline (with and without UNCTAD feature). RMSE, MAE, R^2 .

05_tree_models.ipynb:

- XGBoost + LightGBM. LightGBM native categorical handling for country, INCO term, commodity.
- LightGBM handles NaN in `unctad_adval_rate` natively — no imputation needed.
- Compare vs linear baseline.
- One round `RandomizedSearchCV` on LightGBM. No rabbit-holing.
- Pickle → `models/usaid_model.pkl`.
- Install `shap`. Compute SHAP on 5 sample predictions. Check whether `unctad_adval_rate` appears in top-10 features.

Phase 2 — UNCTAD Feature Analysis + Buffer (Weeks 5–6)

Week 5 — UNCTAD Feature Analysis (~7 hrs)

06_unctad_feature_analysis.ipynb:

- SHAP importance of `unctad_adval_rate` vs other features.
- Partial dependence plot: as UNCTAD benchmark increases, does predicted cost increase? (Sanity check.)
- Residuals by UNCTAD coverage: does the model perform better on rows with direct UNCTAD match vs fallback vs NaN?
- Honest writeup: what does the UNCTAD feature contribute, and what are its limitations (temporal mismatch, aggregation)?

Week 6 — Buffer (~7 hrs)

Non-negotiable buffer. Absorbs whatever broke in Weeks 2–5. If genuinely ahead: refactor `src/models/` into clean `train()` + `predict(inputs)` functions. Write `scripts/train.py` as CLI entry point.

Phase 3 — RAG Layer (Weeks 7–9)*Week 7 — Corpus and Ingestion*

Build `data/corpus/`:

- Incoterms 2020 explainers: ICC public summaries + freight forwarder PDFs (5–10 docs).
- HS code chapter descriptions: WCO public or CSV-to-text.
- Shipping glossary: logistics firm websites, public PDFs.

`src/rag/ingest.py`: parse PDFs → chunk (500 tokens, 50 overlap) → embed (all-MiniLM-L6-v2) → FAISS index → save to disk.

Alt if PDFs chaotic: convert corpus to `.txt` manually once, commit, parse as plain text.

Week 8 — Retriever and Generator

`src/rag/retriever.py`: load FAISS, similarity search, return top-k chunks with source meta-data.

`src/rag/generator.py`: Groq client, prompt template, answer + citations.

```
# src/config.py
import os

LLM_PROVIDER = os.getenv("LLM_PROVIDER", "groq")

def get_llm_client():
    if LLM_PROVIDER == "groq":
        from groq import Groq
        return Groq(api_key=os.getenv("GROQ_API_KEY"))
    elif LLM_PROVIDER == "openai":
        from openai import OpenAI
        return OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

`07_rag_prototype.ipynb`: 10 test queries end-to-end. Log quality. Note failures.

Week 9 — Generative Explanation (SHAP → LLM)

Implement `src/explainer.py`. This is where ML and RAG connect.

Pipeline:

1. User submits form.
2. Model predicts cost. SHAP computes top-5 feature contributions for this prediction.
3. Retriever pulls relevant trade doc chunks.
4. LLM prompt assembled: shipment inputs + SHAP table + retrieved chunks + UNCTAD benchmark for this lane.
5. LLM returns plain-English summary + cites trade docs.

Prompt template:

```

SYSTEM: You are a freight cost analyst specialising in health commodity
supply chains. Explain shipment cost predictions clearly. Always cite
the trade documents provided. Never invent rules not present in context.

USER:
Shipment: {origin} to {destination}, mode: {mode}, weight: {weight}kg,
commodity: {commodity}, INCO term: {inco_term}.

Predicted cost: ${predicted_cost} (range: ${low} -- ${high})

UNCTAD bilateral freight benchmark for {origin} -> {destination} via {mode}:
Ad-valorem rate: {unctad_adval_rate}% (2016-2021 median, structural benchmark)

Top cost drivers (SHAP values, higher = more impact on cost):
{shap_table}

Relevant trade document excerpts:
{retrieved_chunks}

Write:
1. A 3-4 sentence plain-English summary of why this cost was predicted.
2. A bullet list of the top cost drivers with a one-line explanation each.
3. One relevant trade rule or definition from the documents above
   (cite the source).

```

Add query caching (`functools.lru_cache` or dict keyed on input hash).

Phase 4 — What-If Chat and App (Weeks 10–12)

Week 10 — What-If Chat Logic

Implement `src/rag/whatif.py`.

Architecture:

1. **Intent parsing:** LLM call #1 — extract structured modification intent from user message + current shipment context.
2. **Input modification:** apply changes; re-lookup UNCTAD benchmark for new lane if origin/destination/mode changed.
3. **Reprediction:** run modified inputs through model. New cost + SHAP.
4. **RAG retrieval:** retrieve trade doc chunks relevant to the modification.
5. **Response generation:** LLM call #2 — new prediction, delta, explanation, citations.

Intent parsing prompt:

```

SYSTEM: Extract shipment modification intent as JSON. Only output JSON.
No preamble.

USER:
Current shipment: {shipment_dict_as_json}
User message: "{user_message}"

Output JSON with keys:
{
  "modification_type": "mode_change | route_change | weight_change | other",
  "changes": {"field": "new_value"},
  "is_what_if": true | false,
  "rag_only": true | false
}

```

Stateful chat in Streamlit: use `st.session_state` to persist shipment dict and chat history across reruns.

Week 11 — Streamlit App (Local)

`app/streamlit_app.py` — two tabs:

Tab 1 — Predict and Explain:

- Shipment form: origin hub, destination country, mode, weight, commodity, INCO term.
- Predict → cost + uncertainty.
- Explanation: LLM summary + SHAP table + UNCTAD benchmark callout + cited trade context.
- “Open chat about this shipment” → Tab 2, pre-seeded.
- `@st.cache_resource` for model + FAISS index + UNCTAD lookup dict.

Tab 2 — What-If Chat:

- Shows current shipment summary at top.
- Chat input, history via `st.chat_message`.
- Example prompts: “What if I use sea freight?”, “What if destination is Uganda?”, “What does CIF mean?”
- “Reset shipment” button clears session state.

Week 12 — Deploy and Polish

1. Create HF Space (Streamlit), link to GitHub.
2. Add `GROQ_API_KEY` to HF Secrets.
3. Cold start test. Document wait in README.
4. Verify model pickle + FAISS + UNCTAD lookup dict load on HF CPU.
5. Example shipments (one-click “Try this”).
6. Friendly error messages for rate limits.
7. “How it works” collapsible: diagram showing USAID data + UNCTAD enrichment + RAG + LLM.

Phase 5 — Ship (Week 13)*Week 13 — Documentation + Interview Prep*

README 30-second skim test:

1. Problem: health commodity freight cost prediction.
2. Data: USAID transaction records enriched with UNCTAD bilateral freight rate benchmarks.
3. Approach: LightGBM with UNCTAD enrichment, RAG-grounded generative explanation, what-if chat.
4. Results: RMSE/MAE, baseline vs LightGBM, UNCTAD feature SHAP importance.
5. Demo: link + screenshots.
6. Next steps: Olist last-mile dataset as A2, commercial freight data integration.

Interview Q&A (14 samples):

- Why USAID + UNCTAD enrichment instead of two separate ML datasets?
- Why was Brunel dropped? What did the EDA reveal?
- What is the temporal mismatch between USAID and UNCTAD, and how do you handle it?
- Why log-transform the target?
- How does SHAP ground the LLM explanation?
- Why does LightGBM handle the UNCTAD NaN natively, and what does that mean?
- How does the what-if chat re-lookup the UNCTAD benchmark when the lane changes?

- What's the RMSE in USD terms for a typical shipment?
- What happens when the user asks something outside the corpus?
- What are the limitations of using health commodity data for general freight pricing?
- How would you retrain this model on new data in production?
- Why is random train/test split wrong for this data?
- What would you add with 6 more months?
- What's the coverage rate of UNCTAD on USAID lanes, and what's your fallback strategy?

Alternatives Table

Debug max 2 hours before swapping.

Blocker	Primary	Alternative
UNCTAD coverage too low (<30% of USAID rows)	UNCTAD join	Drop UNCTAD feature. USAID-only model. Reframe README: “pure health commodity SC analysis.”
UNCTAD rates uncorrelated with USAID costs	Use as feature	Drop UNCTAD feature. Keep model simple. Still a strong portfolio piece.
USAID too dirty/small	USAID Kaggle	DataCo Smart SC (note: shipping modes are fulfillment tiers, not real freight modes — reframe entirely if used)
Groq rate limits hit during demo	Groq free	together.ai free tier or OpenAI \$5 budget
FAISS install fails (Windows)	faiss-cpu	chromadb — cleaner pip, persistent by default
SHAP slow on LightGBM	<code>shap.TreeExplainer</code>	Gain-based feature importance from LightGBM directly (less accurate but fast)
What-if intent parsing unreliable	Groq LLM parse	Rule-based keyword fallback for common cases (mode_change, route_change)
HF Spaces too slow	HF Spaces	Streamlit Community Cloud
PDF parsing chaotic	pypdf	Convert corpus to .txt manually, commit, parse as plain text
sentence-transformers slow	all-MiniLM-L6-v2	bge-small-en-v1.5 (smaller, faster on CPU)
16GB RAM pressure	float32 embeddings	Cast to float16; FAISS IVF index

Risk Flags

1. Time-based split is mandatory. USAID has a date column. Use time-based split (80/20 at Dec 2013). Random split on temporal data = silent leakage. Will come up in interviews.

2. SHAP grounds the explanation — without it, LLM hallucinates. Do not skip SHAP. An LLM explaining a prediction without being shown the actual model drivers is guessing. This is the most common mistake in “explainable AI” demos and interviewers notice.

3. UNCTAD temporal mismatch must be disclosed. USAID: 2006–2015. UNCTAD: 2016–2021. No year overlap. UNCTAD is a structural benchmark, not a time-matched join. State this explicitly in README and interviews. The defensible framing: bilateral freight rate intensities are structural signals that change slowly; using the 2016–2021 median as a lane benchmark is standard practice in freight gravity-model research.

4. UNCTAD coverage audit is Week 2’s most important output. If coverage is thin, the UNCTAD feature may hurt more than it helps (high NaN rate + weak signal = noisy feature). Run the audit before building the full pipeline. Know your fallback hierarchy: HS4 match → HS2 aggregate → mode-level country-pair average → NaN (LightGBM handles).

5. What-if intent parsing is the hardest piece. LLM-based intent extraction can be flaky. Test with 20+ diverse messages. Build the rule-based fallback from the alternatives table. Two LLM calls per what-if query = double rate limit exposure. Cache aggressively.

6. `st.session_state` in Streamlit. Stateful chat requires careful session state management. Streamlit reruns the entire script on every interaction. The shipment dict and chat history must live in `st.session_state` or they reset. Test before Week 10 build-out.

7. Week 6 buffer is non-negotiable. Do not skip it when feeling ahead in Week 4.

8. Groq TOS. Verify Groq free tier allows public portfolio demo use before launch.

What NOT To Do

- **No deep learning on tabular data.** Red flag in SC interviews.
- **No Streamlit before Week 10.** Model and explanation must be solid first.
- **No Docker.** HF Spaces handles it.
- **No MLflow/W&B.** Mention in README next steps, do not implement.
- **No custom embedding fine-tuning.** Out of scope.
- **No random split if data is temporal.** See Risk Flag 1.
- **Do not let the LLM explain without SHAP input.** See Risk Flag 2.
- **Do not pretend UNCTAD is a time-matched join.** See Risk Flag 3.
- **Do not force UNCTAD into the model if coverage audit fails.** See Risk Flag 4 and Alternatives Table.

Success Criteria

Project A1 is done when all of the following are true:

1. Deployed Streamlit app on HF Spaces, live and accessible via public URL.
2. Tab 1: predicts cost from shipment form; UNCTAD benchmark shown alongside prediction.
3. Tab 1: plain-English explanation with SHAP feature table, UNCTAD callout, and cited trade doc context.
4. Tab 2: chat understands current shipment context from session state.
5. Tab 2: what-if queries rerun model (with updated UNCTAD lookup if lane changed) and return new prediction + delta + explanation.
6. Tab 2: general trade questions answered via RAG with citations.
7. GitHub repo has clean README with results table (RMSE/MAE baseline vs LightGBM), UNCTAD coverage stats, demo link, architecture description, two screenshots.
8. You can answer 14 interview questions about this project without notes, including the UNCTAD temporal mismatch question.
9. LightGBM beats linear baseline on RMSE.
10. OR: UNCTAD feature dropped after coverage audit fails — USAID-only model shipped, README updated honestly.

Next checkpoint: Week 2 — post UNCTAD coverage audit results: what % of USAID rows get direct match, what % fallback, what % NaN. Post correlation between UNCTAD rates and USAID actual costs. Go/no-go decision on UNCTAD feature before building pipeline.